

Mount Kenya University



Unlocking Infinite Possibilities

NAME : STEPHEN KIMANI

ADMISSION: BSCCS/2024/57444

COURSE : BACHELOR OF SCIENCE IN COMPUTER SCIENCE

UNIT CODE: BIT 4138

UNIT NAME: ADVANCED CRYPTOGRAPHY

LECTURER: MICHAEL NYORO

SEMESTER /ACADEMIC YEAR: SEMESTER 6

**PROJECT DETAILS: IMPLEMENTATION AND ANALYSIS OF
CRYPTOGRAPHIC ALGORITHMS USING PYTHON**

Technologies Used:

- I. Python 3**
- II. Visual Studio Code**
- III. Git**
- IV. OpenSSL**
- V. PyCryptodome**
- VI. Windows PowerShell / Command Prompt**

ONLINE PORTFOLIO/ GITHUB LINK

<https://github.com/Kimani985/BIT4138-Advanced-Cryptography>

Contents

WEEK 1: Cryptography Environment Setup.....	6
Installation and Configuration of Cryptography Development Environment.....	6
<i>Figure 1: Successful installation of Python and Visual Studio Code for developing and testing cryptographic applications</i>	6
Figure 2: Installation of cryptography libraries using pip, enabling implementation of encryption and decryption algorithms.	7
Figure 3: OpenSSL version verification confirming successful configuration of cryptographic command-line tools.....	7
Figure 4: Execution of a Python test script verifying the development environment is operational.	8
Figure 5: Initialization of GitHub repository for version control and project documentation.	9
CREATIVE TASK - Environment Checker	9
Figure 6: Automated environment verification tool displaying the installed versions of Python, Git, and OpenSSL to confirm system readiness for cryptographic development.	9
Reflection.....	10
WEEK 2: Classical Cryptography and Cipher Design.....	11
Implementation of Classical Encryption Algorithms	11
Figure 7: Caesar Cipher implementation demonstrating character shifting using a fixed encryption key.....	11
Figure 8: Successful encryption and decryption outputs generated by the implemented cipher algorithm.	12
Figure 9: Input validation process preventing invalid plaintext entries during encryption.	13
Figure 10: Vigenère Cipher implementation demonstrating keyword-based encryption.	14
Figure 11: Cipher testing results obtained using different messages and encryption keys.	14
CREATIVE TASK - Cryptography Toolkit	14

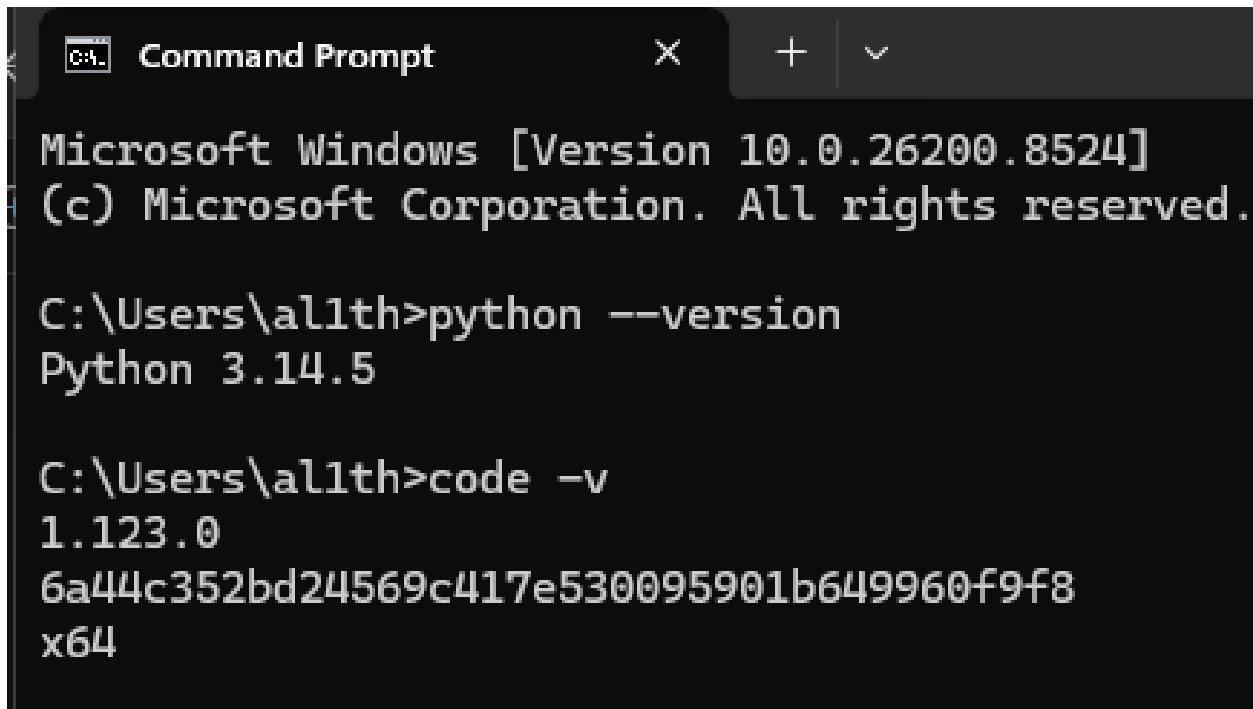
Figure 12: Interactive cryptography toolkit providing a menu-driven interface for selecting and executing classical encryption algorithms.	14
Reflection.....	14
WEEK 3: Stream Ciphers and Randomness Testing.....	15
Implementation of Stream Cipher Systems and Random Sequence Analysis	15
Figure 13: Implementation of a Linear Feedback Shift Register (LFSR) used to generate pseudorandom sequences.	15
Figure 14: Generated pseudorandom binary sequence produced by the implemented LFSR.	15
Figure 15: Statistical randomness testing results showing distribution of generated binary values.	16
Figure 16: RC4-style stream cipher simulation demonstrating encryption using a generated keystream.	16
Figure 17: Performance evaluation results showing execution time during encryption operations.	17
CREATIVE TASK- Randomness Challenge Game	17
Figure 18: Interactive Randomness Challenge Game demonstrating binary sequence generation, user prediction, and statistical analysis of pseudorandom output.	17
Reflection.....	17
WEEK 4: Block Cipher Design and AES.....	18
Advanced Encryption Standard (AES) and Block Cipher Operations	18
Figure 19: AES encryption program implementing symmetric key cryptography for secure file protection	18
Figure 20: AES symmetric key generation process used for encryption and decryption operations	19
Figure 21: Successful encryption of a text file using the generated AES key.	19
Figure 22: Successful decryption process restoring the original file contents.	19
<i>Figure 23: AES performance evaluation showing execution time during encryption....</i>	<i>20</i>
CREATIVE TASK - Secure File Locker.....	20
Figure 24: Secure File Locker demonstrating user-driven AES encryption and successful recovery of encrypted file contents through decryption.	20
Reflection.....	20

WEEK 5: Public Key Cryptography (RSA)	21
Implementation of RSA encryption and Key Management.....	21
Figure 25: Generation of RSA public and private key pairs used for secure communication.	21
Figure 26: Encryption of plaintext using the RSA public key.	21
Figure 27: Successful recovery of the original message using the RSA private key.....	22
Figure 28: Simulation of secure communication using RSA public and private key cryptography.....	22
Figure 29: RSA testing and validation confirming successful encryption and decryption operations.	22
CREATIVE TASK - Secure RSA Messenger	23
Figure 30: Secure RSA Messenger demonstrating encrypted communication between Alice and Bob through RSA public-key encryption and private-key decryption.....	23
Reflection.....	23
WEEK 6 - SPN, S-Boxes and Non-Linearity.....	24
Class Activity 1: Manual SPN Simulation	24
Figure 31: Manual SPN simulation showing substitution and permutation across two rounds.....	24
Class Activity 2 – S-Box Analysis.....	24
Figure 32 : S-Box analysis showing mapping uniqueness and security evaluation.....	24
Figure 33: Feistel and SPN structure comparison.....	25
Implementation of a Simple Substitution-Permutation Network (SPN)	25
Figure 34: Simple SPN cipher implementation and execution.	25
CREATIVE TASK 1 - SPN Visualizer	26
Figure 35: SPN Visualizer illustrating the step-by-step transformation of plaintext through substitution and permutation stages to produce ciphertext.....	26
Creative Task 2 - Avalanche Effect Analyzer	26
Figure 36: Avalanche Effect Analyzer showing input difference analysis.	26
REFLECTION	27
WEEK 7 - Cryptanalysis Techniques	28
Class Activity 1: Difference Analysis	28

Figure 37:Difference analysis implementation and execution.	28
Class Activity 2: Frequency Analysis	28
Figure 38 : Frequency analysis showing character occurrence counts.	28
Class Activity 3: Probability Analysis	29
Figure 39: Probability analysis showing likelihood calculation.	29
Class Activity 4: Differential Cryptanalysis Simulation	30
Figure 40:Differential cryptanalysis simulation showing plaintext differences.	30
CREATIVE TASK – Cryptanalysis Toolkit	31
Figure 41: Cryptanalysis Toolkit integrating difference, frequency, and probability analysis techniques.	31
REFLECTION.....	31

WEEK 1: Cryptography Environment Setup

Installation and Configuration of Cryptography Development Environment



```
Microsoft Windows [Version 10.0.26200.8524]
(c) Microsoft Corporation. All rights reserved.

C:\Users\allth>python --version
Python 3.14.5

C:\Users\allth>code -v
1.123.0
6a44c352bd24569c417e530095901b649960f9f8
x64
```

Figure 1: Successful installation of Python and Visual Studio Code for developing and testing cryptographic applications.

```
C:\Users\al1th>pip install cryptography
Collecting cryptography
  Downloading cryptography-48.0.0-cp311-abi3-win_amd64.whl.metadata (4.3 kB)
Collecting cffi>=2.0.0 (from cryptography)
  Downloading cffi-2.0.0-cp314-cp314-win_amd64.whl.metadata (2.6 kB)
Collecting pycparser (from cffi>=2.0.0->cryptography)
  Downloading pycparser-3.0-py3-none-any.whl.metadata (8.2 kB)
  Downloading cryptography-48.0.0-cp311-abi3-win_amd64.whl (3.8 MB)
    3.8/3.8 MB 3.6 MB/s 0:00:01
  Downloading cffi-2.0.0-cp314-cp314-win_amd64.whl (185 kB)
  Downloading pycparser-3.0-py3-none-any.whl (48 kB)
Installing collected packages: pycparser, cffi, cryptography
Successfully installed cffi-2.0.0 cryptography-48.0.0 pycparser-3.0

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: C:\Users\al1th\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip

C:\Users\al1th>pip install pycryptodome
Collecting pycryptodome
  Downloading pycryptodome-3.23.0-cp37-abi3-win_amd64.whl.metadata (3.5 kB)
  Downloading pycryptodome-3.23.0-cp37-abi3-win_amd64.whl (1.8 MB)
    1.8/1.8 MB 863.2 kB/s 0:00:02
Installing collected packages: pycryptodome
Successfully installed pycryptodome-3.23.0

[notice] A new release of pip is available: 26.1.1 -> 26.1.2
[notice] To update, run: C:\Users\al1th\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip

C:\Users\al1th>pip list
Package      Version
-----
cffi         2.0.0
cryptography 48.0.0
pip          26.1.1
pycparser    3.0
```

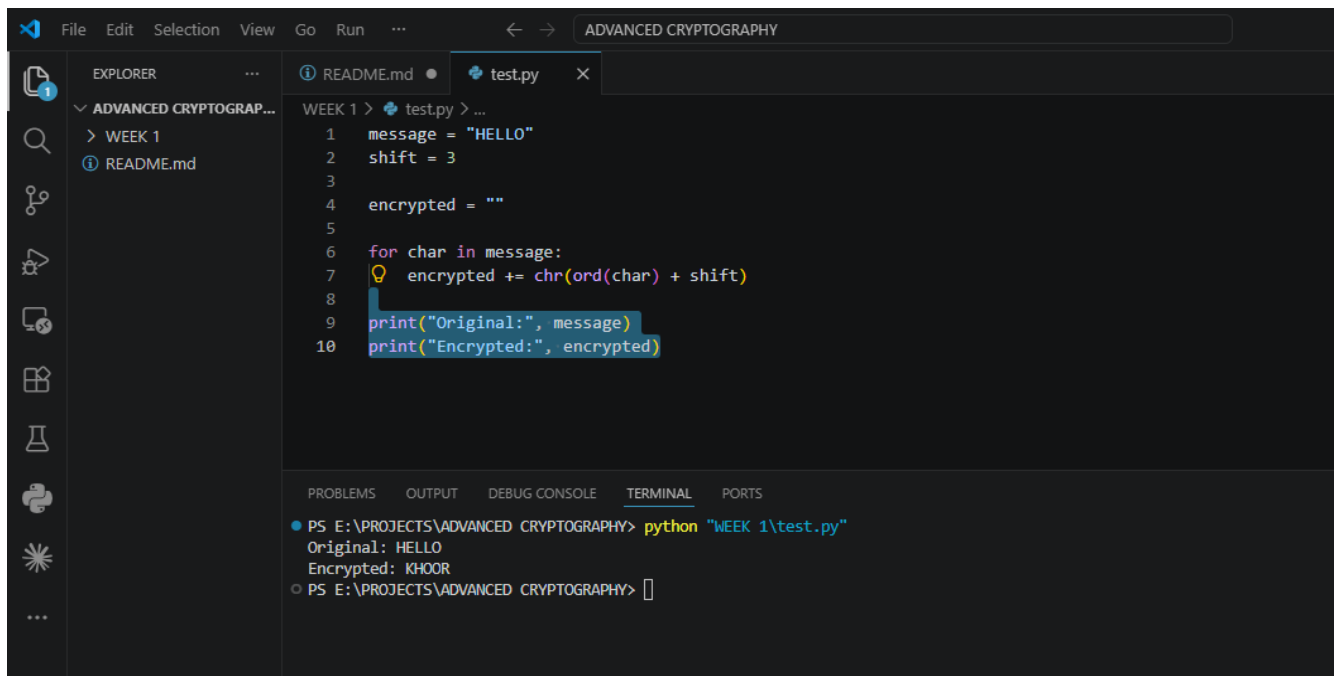
Figure 2: Installation of cryptography libraries using pip, enabling implementation of encryption and decryption algorithms.

```
MINGW64:/c/Users/al1th

al1th@HIM MINGW64 ~
$ openssl version
OpenSSL 3.5.6 7 Apr 2026 (Library: OpenSSL 3.5.6 7 Apr 2026)

al1th@HIM MINGW64 ~
$ |
```

Figure 3: OpenSSL version verification confirming successful configuration of cryptographic command-line tools.



The screenshot shows the Visual Studio Code interface with a project named "ADVANCED CRYPTOGRAPHY". The Explorer sidebar on the left shows the project structure with "WEEK 1" and "README.md". The main editor window displays a Python file named "test.py" with the following code:

```
1 message = "HELLO"
2 shift = 3
3
4 encrypted = ""
5
6 for char in message:
7     encrypted += chr(ord(char) + shift)
8
9 print("Original:", message)
10 print("Encrypted:", encrypted)
```

The bottom panel shows the "TERMINAL" tab with the following output:

```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> python "WEEK 1\test.py"
Original: HELLO
Encrypted: KHOOR
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 4: Execution of a Python test script verifying the development environment is operational.

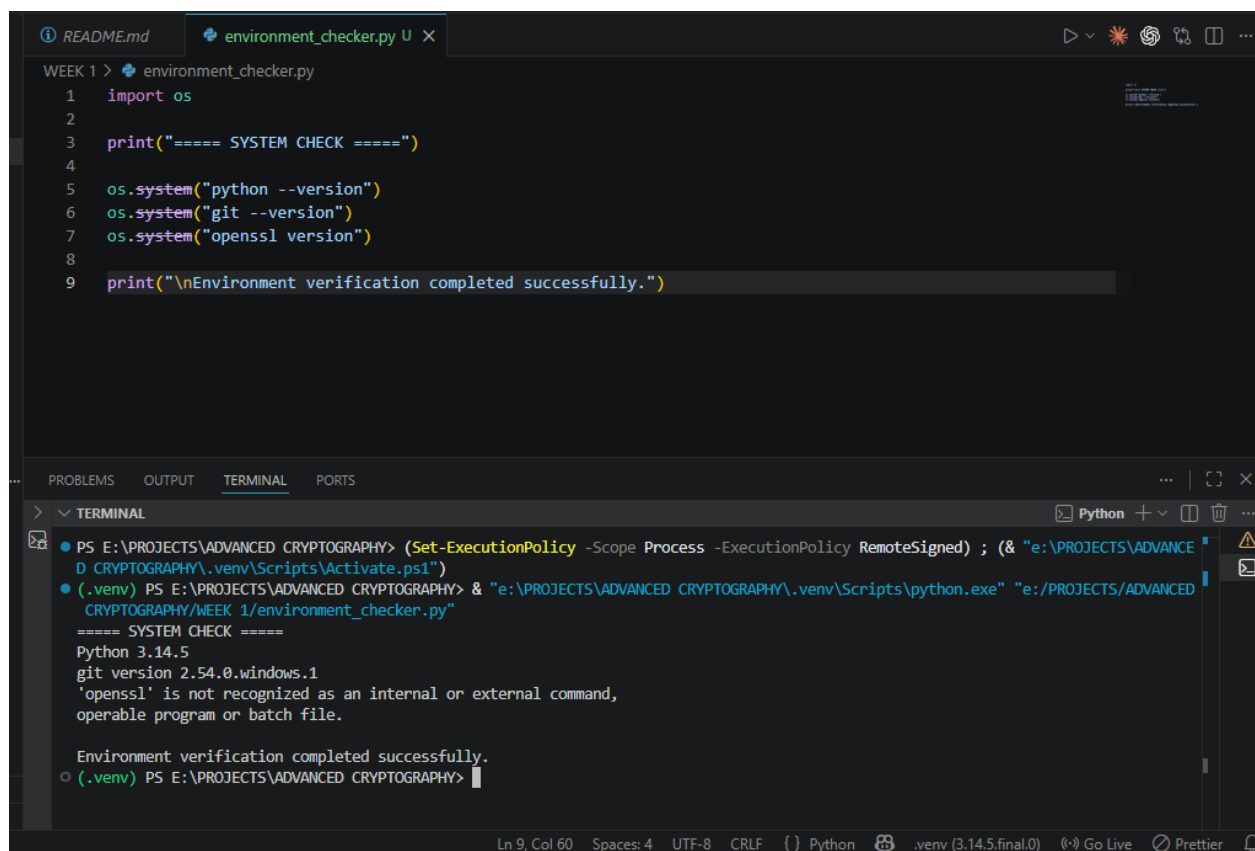

```

PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> git init
Initialized empty Git repository in E:/PROJECTS/ADVANCED CRYPTOGRAPHY/.git/
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> git add .
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> git commit -m "Week 1 environment setup completed"
[main (root-commit) 2652837] Week 1 environment setup completed
 2 files changed, 10 insertions(+)
 create mode 100644 README.md
 create mode 100644 WEEK 1/test.py
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>

```

Figure 5: Initialization of GitHub repository for version control and project documentation.

CREATIVE TASK - Environment Checker



```

WEEK 1 > environment_checker.py
1  import os
2
3  print("==== SYSTEM CHECK =====")
4
5  os.system("python --version")
6  os.system("git --version")
7  os.system("openssl version")
8
9  print("\nEnvironment verification completed successfully.")

```

```

PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 1/environment_checker.py"
==== SYSTEM CHECK =====
Python 3.14.5
git version 2.54.0.windows.1
'openssl' is not recognized as an internal or external command,
operable program or batch file.

Environment verification completed successfully.
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>

```

Figure 6: Automated environment verification tool displaying the installed versions of Python, Git, and OpenSSL to confirm system readiness for cryptographic development.

Reflection

This week focused on setting up the cryptography development environment. Python, VS Code, OpenSSL, and required libraries were successfully installed and tested. GitHub was configured for version control. The environment is now ready for implementing and testing cryptographic algorithms.

WEEK 2: Classical Cryptography and Cipher Design

Implementation of Classical Encryption Algorithms

```
WEEK 2 > 📄 classical_cipher.py > ...
1  def caesar_encrypt(text, shift):
2      result = ""
3
4      for char in text:
5          if char.isalpha():
6              result += chr((ord(char.upper()) - 65 + shift) % 26 + 65)
7          else:
8              result += char
9
10     return result
11
12
13 def caesar_decrypt(text, shift):
14     return caesar_encrypt(text, -shift)
15
16
17 message = input("Enter message: ")
18
19 if not message.replace(" ", "").isalpha():
20     print("Invalid input! Use letters only.")
21 else:
22     shift = int(input("Enter shift value: "))
23
24     encrypted = caesar_encrypt(message, shift)
25     decrypted = caesar_decrypt(encrypted, shift)
26
27     print("\nOriginal:", message)
28     print("Encrypted:", encrypted)
29     print("Decrypted:", decrypted)
```

Figure 7: Caesar Cipher implementation demonstrating character shifting using a fixed encryption key.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> cd "WEEK 2"
● PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2> python classical_cipher.py
Enter message: HELLO
Enter shift value: 3

Original: HELLO
Encrypted: KHOOR
Decrypted: HELLO
○ PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2> |
```

Figure 8: Successful encryption and decryption outputs generated by the implemented cipher algorithm.

```
● PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2> python classical_cipher.py
Enter message: HELLO123
Invalid input! Use letters only.
○ PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2> |
```

Figure 9: Input validation process preventing invalid plaintext entries during encryption.

```
WEEK 2 > vigenere_cipher.py > ...
1  def caesar_encrypt(text, shift):
2      result = ""
3
4      for char in text:
5          if char.isalpha():
6              result += chr((ord(char.upper()) - 65 + shift) % 26 + 65)
7          else:
8              result += char
9
10     return result
11
12
13  def vigenere_encrypt(text, key):
14      result = ""
15      key = key.upper()
16
17      key_index = 0
18
19      for char in text.upper():
20          if char.isalpha():
21              shift = ord(key[key_index % len(key)]) - 65
22              result += chr((ord(char) - 65 + shift) % 26 + 65)
23              key_index += 1
24          else:
25              result += char
26
27      return result
28
29
30  message = input("Enter message: ")
31
32  if not message.replace(" ", "").isalpha():
```

Figure 10: Vigenère Cipher implementation demonstrating keyword-based encryption.

```
● PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> python "WEEK 2\vigenere_cipher.py"
Enter message: SECURITY

Caesar Cipher: VHFXULWB
Enter Vigenere key: LAMP
Vigenere Cipher: DEOJCIFN
○ PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 11: Cipher testing results obtained using different messages and encryption keys.

CREATIVE TASK - Cryptography Toolkit

```
● (.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> python crypto_toolkit.py
E:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe: can't open file 'E:\PROJECTS\ADVANCED CRYPTOGRAPHY\crypto_toolkit.py':
[Errno 2] No such file or directory
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> cd "E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2"
● >> python crypto_toolkit.py

===== CRYPTOGRAPHY TOOLKIT =====
1. Caesar Encrypt
2. Caesar Decrypt
3. Vigenère Encrypt
4. Exit
Enter your choice: 1
Enter message: HELLO
Enter shift value: 3

Encrypted Message: KHOOR

===== CRYPTOGRAPHY TOOLKIT =====
1. Caesar Encrypt
2. Caesar Decrypt
3. Vigenère Encrypt
4. Exit
Enter your choice: 4

Thank you for using the Cryptography Toolkit.
○ (.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2>
```

Figure 12: Interactive cryptography toolkit providing a menu-driven interface for selecting and executing classical encryption algorithms.

Reflection

The implementation of Caesar and Vigenère ciphers demonstrated the importance of encryption keys in protecting information. Testing showed that Vigenère Cipher provides stronger security through keyword-based encryption, while Caesar Cipher remains vulnerable to simple cryptanalysis techniques.

WEEK 3: Stream Ciphers and Randomness Testing

Implementation of Stream Cipher Systems and Random Sequence Analysis

```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 3> python randomness.py
===== LFSR SIMULATION =====
Generated Sequence:
```

Figure 13: Implementation of a Linear Feedback Shift Register (LFSR) used to generate pseudorandom sequences.

[illegible]

Figure 14: Generated pseudorandom binary sequence produced by the implemented LFSR.

```
===== RANDOMNESS TEST =====  
Number of Ones : 58  
Number of Zeros: 42  
Percentage of Ones : 58.00%  
Percentage of Zeros: 42.00%  
Result: Weak Randomness
```

Figure 15: Statistical randomness testing results showing distribution of generated binary values.

```
===== RC4 STYLE SIMULATION =====  
Message: HELLO  
Key Stream: [153, 253, 3, 156, 58]  
Encrypted: [209, 184, 79, 208, 117]
```

Figure 16: RC4-style stream cipher simulation demonstrating encryption using a generated keystream.

```
===== PERFORMANCE RESULTS =====  
Execution Time: 7.724761962890625e-05 seconds  
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 3>
```


Figure 17: Performance evaluation results showing execution time during encryption operations.

CREATIVE TASK- Randomness Challenge Game

```
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 2> cd "E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 3"
(.venv) PS E:\PRO\python randomness_game.pyHY\WEEK 3>
===== RANDOMNESS CHALLENGE GAME =====
Enter sequence length: 20

Sequence Generated Successfully!
Guess how many 1's are in the sequence: 10

===== RESULTS =====
Generated Sequence:
[1, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0]

Number of Ones : 12
Number of Zeros: 8

❌ Wrong guess.
Difference: 2

===== RANDOMNESS ANALYSIS =====
Percentage of Ones : 60.00%
Percentage of Zeros: 40.00%

Program Completed Successfully.
```

Figure 18: Interactive Randomness Challenge Game demonstrating binary sequence generation, user prediction, and statistical analysis of pseudorandom output.

Reflection

This week focused on pseudorandom sequence generation and stream cipher concepts. An LFSR was implemented to generate binary sequences, and statistical testing was performed to evaluate randomness. An RC4-style simulation demonstrated how stream ciphers use keystreams to encrypt data efficiently.

WEEK 4: Block Cipher Design and AES

Advanced Encryption Standard (AES) and Block Cipher Operations

```
WEEK 4 > aes_encryption.py > ...
1  from Crypto.Cipher import AES
2  from Crypto.Random import get_random_bytes
3  import time
4
5  print("==== AES ENCRYPTION SYSTEM ====")
6
7  # Generate AES key
8  key = get_random_bytes(16)
9
10 print("\nGenerated AES Key:")
11 print(key.hex())
12
13 # Read file
14 with open("secret.txt", "rb") as file:
15     data = file.read()
16
17 # Padding
18 while len(data) % 16 != 0:
19     data += b' '
20
21 cipher = AES.new(key, AES.MODE_ECB)
22
23 start = time.time()
24
25 encrypted_data = cipher.encrypt(data)
26
27 with open("encrypted.bin", "wb") as file:
28     file.write(encrypted_data)
29
30 end = time.time()
31
32 print("\nFile Encrypted Successfully")
```

Figure 19: AES encryption program implementing symmetric key cryptography for secure file protection

Generated AES Key:
232507cdefddad9009c2f250fb936109

Figure 20: AES symmetric key generation process used for encryption and decryption operations

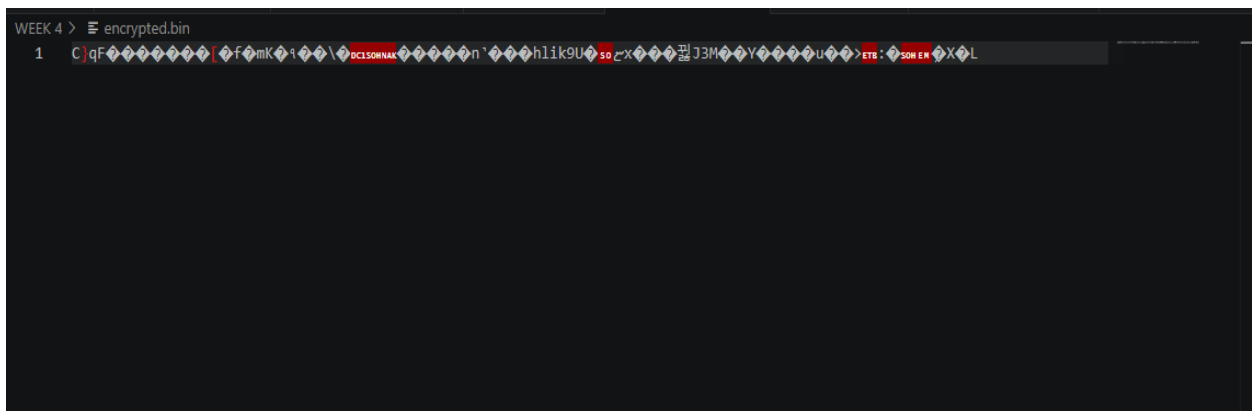


Figure 21: Successful encryption of a text file using the generated AES key.

Recovered Text:
This is a confidential BIT4138 file.
AES encryption is being tested.

Figure 22: Successful decryption process restoring the original file contents.

```

===== PERFORMANCE RESULTS =====
Encryption Time: 0.00034737586975097656 seconds
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 4>

```

Figure 23: AES performance evaluation showing execution time during encryption.

CREATIVE TASK - Secure File Locker

```

PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 4\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> cd "WEEK 4"
>> python secure_file_locker.py
===== SECURE FILE LOCKER =====
Enter file name to encrypt: secret.txt
File encrypted successfully!
Decrypt file now? (Y/N): y
File decrypted successfully!
Recovered file saved as recovered.txt
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 4>

```

Figure 24: Secure File Locker demonstrating user-driven AES encryption and successful recovery of encrypted file contents through decryption.

Reflection

AES encryption was implemented to protect files using symmetric cryptography. Secure key generation, encryption, and decryption processes were successfully tested. The practical activity demonstrated the importance of key management and the effectiveness of AES in securing sensitive information.

WEEK 5: Public Key Cryptography (RSA)

Implementation of RSA encryption and Key Management

```
PTOGRAPHY\.venv\Scripts\Activate.ps1")
• (.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 5> python rsa_encryption.py
• (.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 5> python rsa_encryption.py
===== RSA ENCRYPTION SYSTEM =====

===== KEY GENERATION =====
Public Key Generated
Private Key Generated
Original Message:
```

Figure 25: Generation of RSA public and private key pairs used for secure communication.

```
===== PUBLIC KEY ENCRYPTION =====
Encrypted Message:
490faeb3e02d3bf10169d815bad4c25b32f2cb4cadcc2a0170d2553eb032757f76f750955230f487623fcec2f55f5c859be2be4dad41c432d89fcfdf81ec890f439a287324773
c1c6cca3dfc0873e9de2cd24084e65df00fbd3b4b6bd469ca1e0f863043ff70e22a884a19fdb5b7a947e3556413fb0018db53d3dbdcf713ce443829c121b967c42a6848214bc7
495ae97d3909b841e41a33440c8e3622c70bcccc7a507c8fa0031c15cfc7352c01608a580a0b1b9142ac06eb23f559231773e8e16f6e94841d01a9bc10b1b9aba679d40656e6
decdfca9c89dd3485bf766187cb407443bb60fd7b96ef178877f547c49d6cb9384e5fe969fdb6ab047ec7
```

Figure 26: Encryption of plaintext using the RSA public key.

```
===== PRIVATE KEY DECRYPTION =====
Recovered Message:
HELLO BIT4138
```

Figure 27: Successful recovery of the original message using the RSA private key.

```
===== SECURE MESSAGE TRANSMISSION =====  
Alice encrypts with Bob's public key  
Bob decrypts with his private key
```

Figure 28: Simulation of secure communication using RSA public and private key cryptography.

```
===== RSA VALIDATION =====  
Validation Successful  
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 5>
```

Figure 29: RSA testing and validation confirming successful encryption and decryption operations.

CREATIVE TASK - Secure RSA Messenger

```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> cd "WEEK 5"
>> python secure_messenger.py
===== SECURE RSA MESSENGER =====
Alice's Message: Hello Bob

Encrypted Message:
476d5677cd714cd46887dce15cdad21f29d3db89ebff2f751487a20f38b868604df98ac2f0af8e99b7b8ead403d8a870cfc605319fd3835c5c5a4b6854df6a936ada61822b7f
448a1e806a71c713a46496f2a82fc72028cef609cbf10f8a0b533c618287d3047aaf82f32d9c221986f33d1958f08841de74bfac9dac74b4731967aef6383311bd6e241c-f1b2
faed94f2e0c87d48cb6167ac832d4103f5f88314d88736bf7c6f27e440c0a71b5e98a195dd6c92c5943d16800d9ff9b2efffce9ec5a244b63815ae1206769468742290c38a10
0285b5674b7293ba6ec8c9619cc377066e85132e11b8956f87165562b3f31edb955d3a1174fdb14f2cec1e898e4

Bob Received:
Hello Bob

Secure transmission completed successfully.
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 5>
```

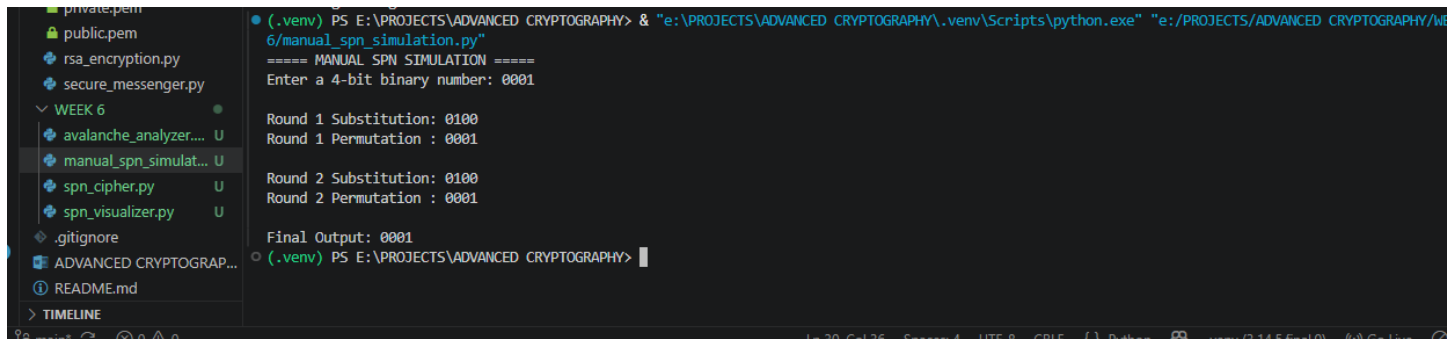
Figure 30: Secure RSA Messenger demonstrating encrypted communication between Alice and Bob through RSA public-key encryption and private-key decryption.

Reflection

RSA encryption demonstrated the use of public and private keys for secure communication. The Secure RSA Messenger successfully encrypted and decrypted messages, highlighting the practical application of asymmetric cryptography.

WEEK 6 - SPN, S-Boxes and Non-Linearity

Class Activity 1: Manual SPN Simulation



```
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 6\manual_spn_simulation.py"
===== MANUAL SPN SIMULATION =====
Enter a 4-bit binary number: 0001

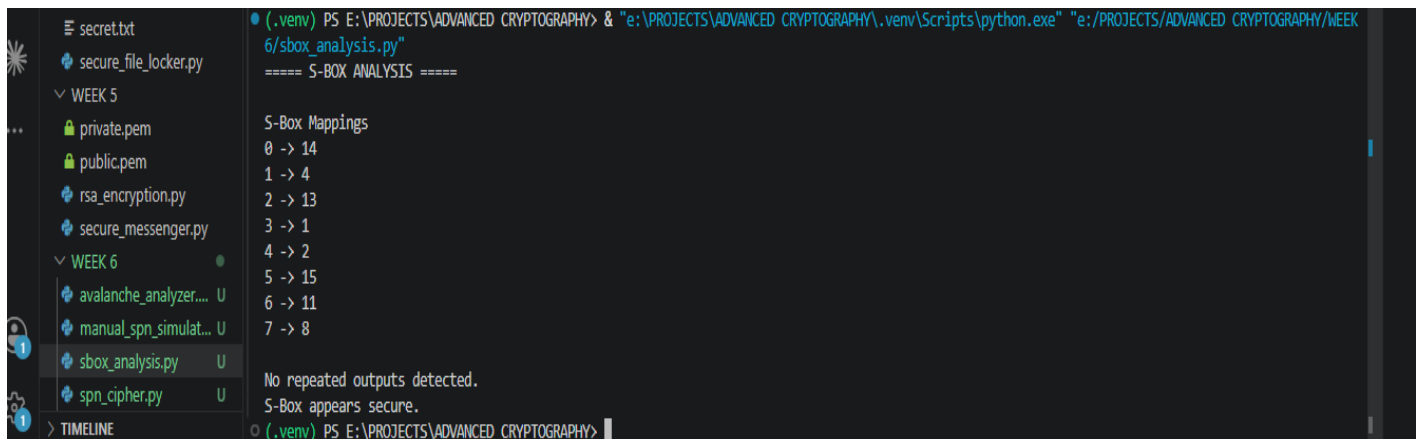
Round 1 Substitution: 0100
Round 1 Permutation : 0001

Round 2 Substitution: 0100
Round 2 Permutation : 0001

Final Output: 0001
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 31: Manual SPN simulation showing substitution and permutation across two rounds.

Class Activity 2 – S-Box Analysis



```
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 6\sbox_analysis.py"
===== S-BX ANALYSIS =====

S-Box Mappings
0 -> 14
1 -> 4
2 -> 13
3 -> 1
4 -> 2
5 -> 15
6 -> 11
7 -> 8

No repeated outputs detected.
S-Box appears secure.
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 32 : S-Box analysis showing mapping uniqueness and security evaluation


```
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 6/feistel_vs_spn.py"
===== FEISTEL VS SPN COMPARISON =====

FEISTEL NETWORK
Structure : Splits data into halves
Security : Good
Complexity: Moderate
Example : DES

SPN NETWORK
Structure : Uses entire block
Security : Very High
Complexity: Higher
Example : AES

Conclusion:
SPN provides stronger security and is used by AES.
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 33: Feistel and SPN structure comparison.

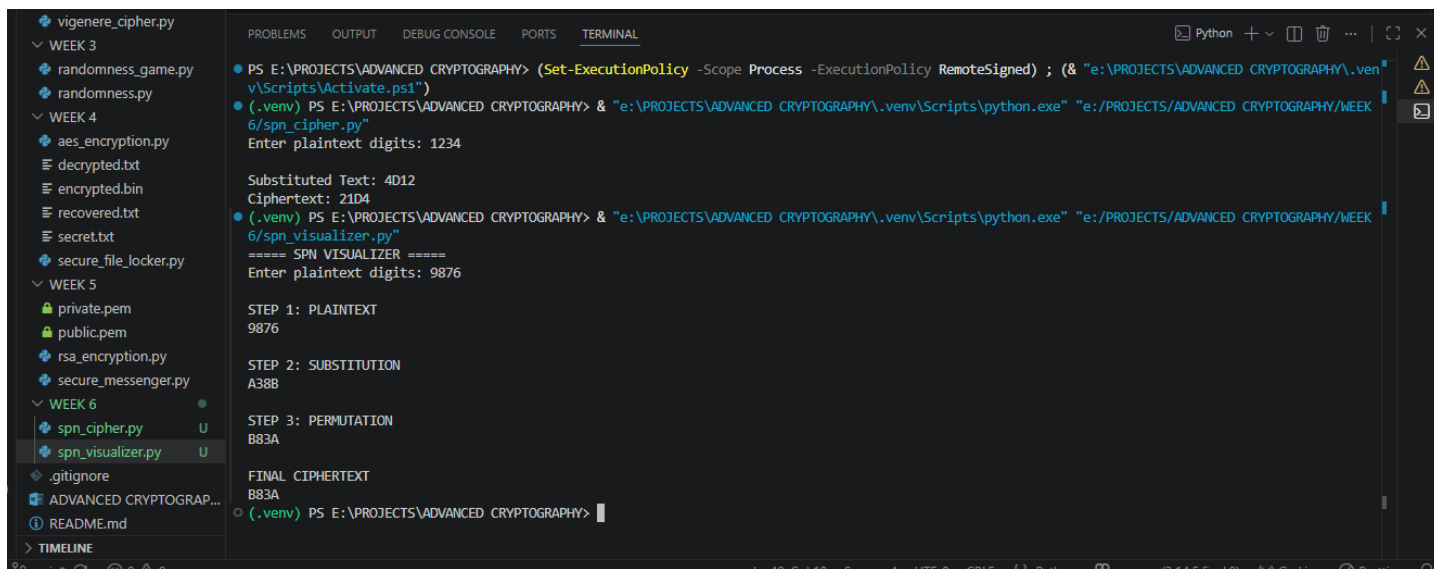
Implementation of a Simple Substitution-Permutation Network (SPN)

```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 6/spn_cipher.py"
Enter plaintext digits: 1234

Substituted Text: 4D12
Ciphertext: 21D4
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 34: Simple SPN cipher implementation and execution.

CREATIVE TASK 1 - SPN Visualizer



```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 6/spn_cipher.py"
Enter plaintext digits: 1234

Substituted Text: 4D12
Ciphertext: 21D4
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 6/spn_visualizer.py"
===== SPN VISUALIZER =====
Enter plaintext digits: 9876

STEP 1: PLAINTEXT
9876

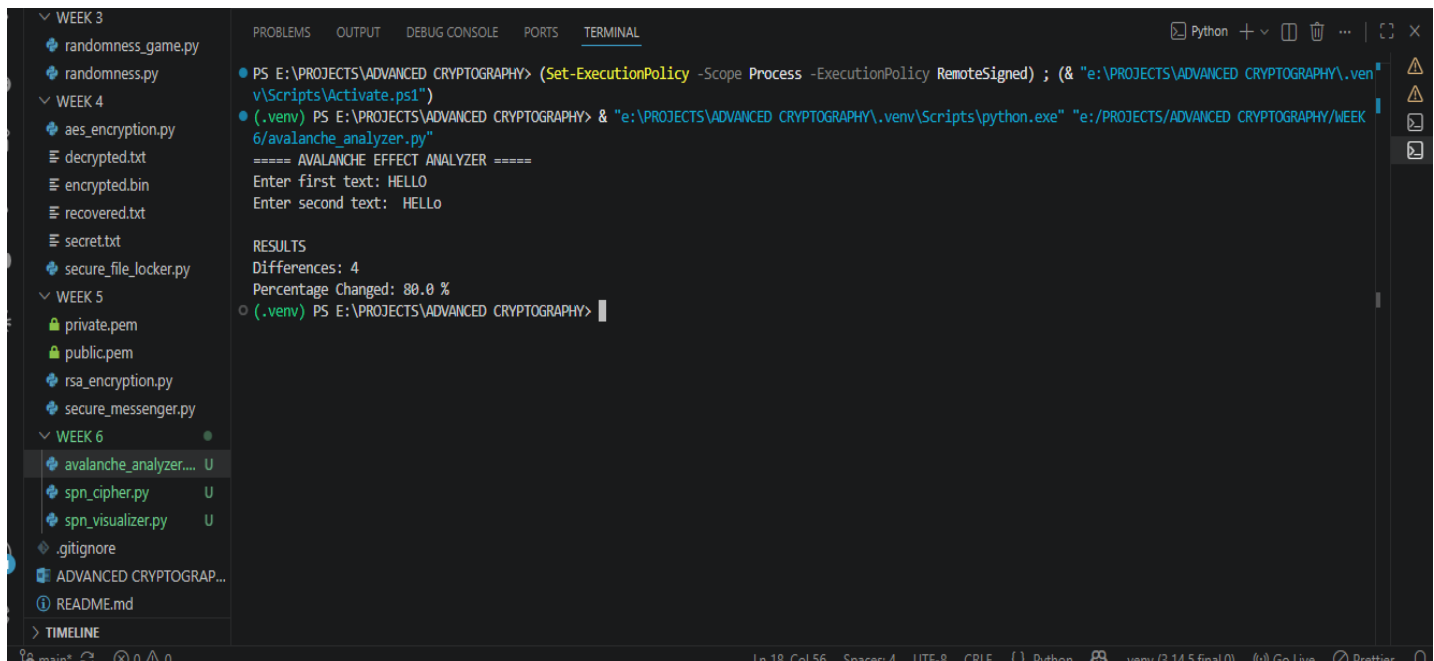
STEP 2: SUBSTITUTION
A38B

STEP 3: PERMUTATION
B83A

FINAL CIPHERTEXT
B83A
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 35: SPN Visualizer illustrating the step-by-step transformation of plaintext through substitution and permutation stages to produce ciphertext.

Creative Task 2 - Avalanche Effect Analyzer



```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 6/avalanche_analyzer.py"
===== AVALANCHE EFFECT ANALYZER =====
Enter first text: HELLO
Enter second text: HELlo

RESULTS
Differences: 4
Percentage Changed: 80.0 %
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

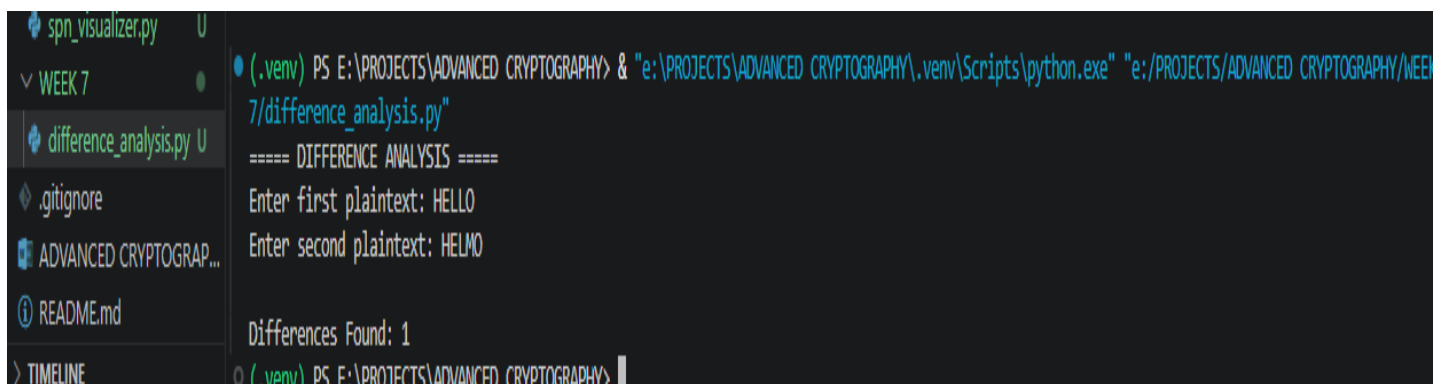
Figure 36: Avalanche Effect Analyzer showing input difference analysis.

REFLECTION

Week 6 explored Substitution-Permutation Networks through manual simulations, S-Box analysis, and SPN implementation. The creative tasks enhanced understanding of cipher structure and demonstrated how small input changes can influence cryptographic outputs.

WEEK 7 - Cryptanalysis Techniques

Class Activity 1: Difference Analysis

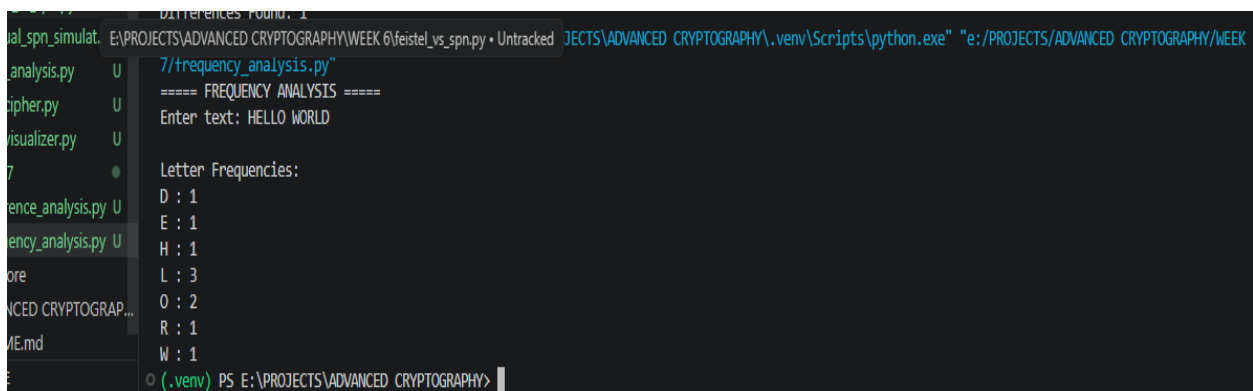


```
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 7/difference_analysis.py"
===== DIFFERENCE ANALYSIS =====
Enter first plaintext: HELLO
Enter second plaintext: HELMO

Differences Found: 1
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 37: Difference analysis implementation and execution.

Class Activity 2: Frequency Analysis

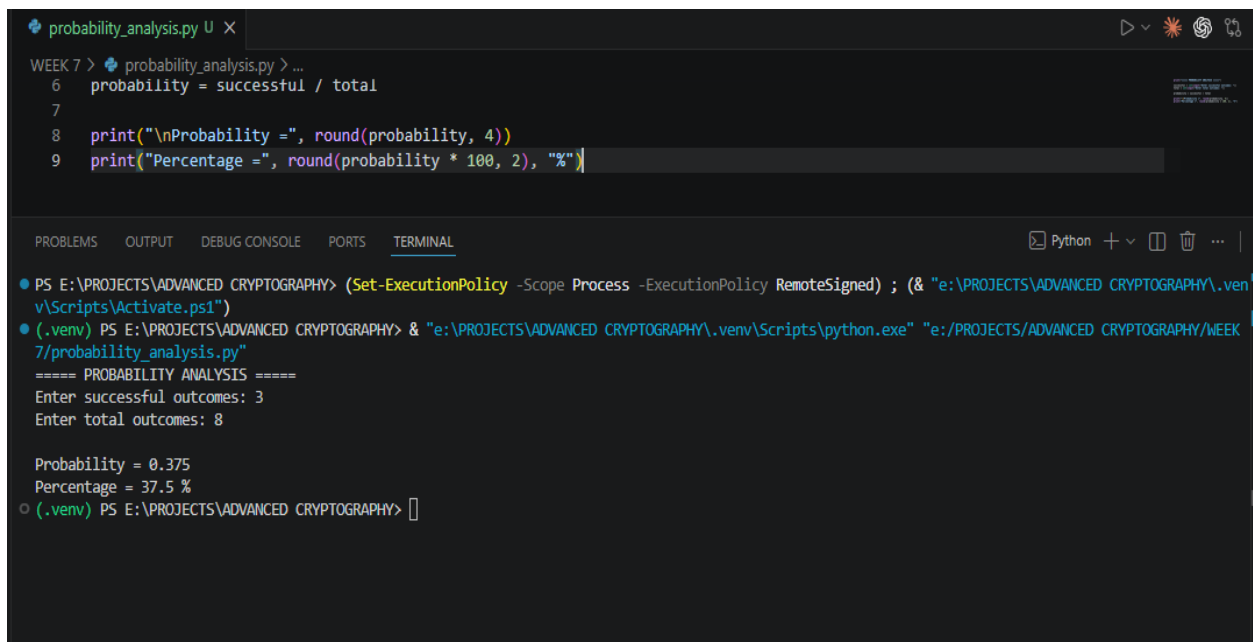


```
E:\PROJECTS\ADVANCED CRYPTOGRAPHY\WEEK 6\feistel_vs_spn.py • Untracked JECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 7/frequency_analysis.py"
===== FREQUENCY ANALYSIS =====
Enter text: HELLO WORLD

Letter Frequencies:
D : 1
E : 1
H : 1
L : 3
O : 2
R : 1
W : 1
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 38 : Frequency analysis showing character occurrence counts.

Class Activity 3: Probability Analysis



The image shows a Visual Studio Code editor window with a Python file named `probability_analysis.py` open. The code in the file is as follows:

```
WEEK 7 > probability_analysis.py > ...
6 probability = successful / total
7
8 print("\nProbability =", round(probability, 4))
9 print("Percentage =", round(probability * 100, 2), "%")
```

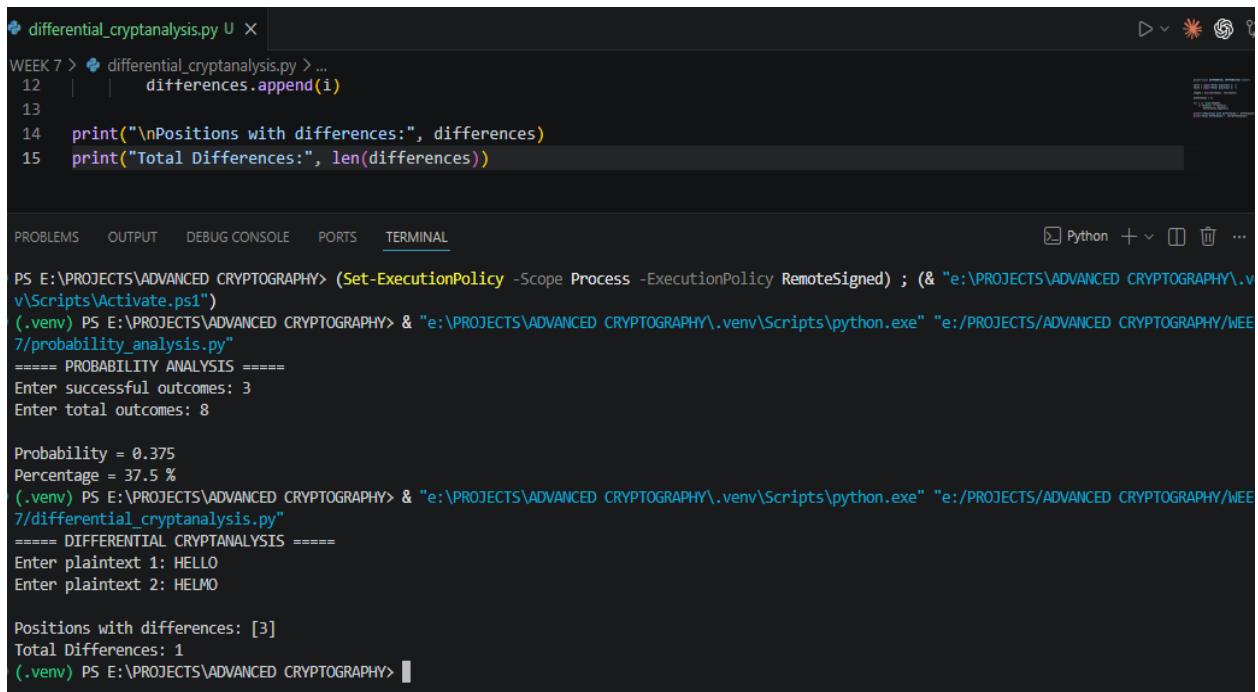
Below the code editor, the `TERMINAL` tab is active, showing the execution of the script. The terminal output is:

```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 7/probability_analysis.py"
===== PROBABILITY ANALYSIS =====
Enter successful outcomes: 3
Enter total outcomes: 8

Probability = 0.375
Percentage = 37.5 %
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 39: Probability analysis showing likelihood calculation.

Class Activity 4: Differential Cryptanalysis Simulation



The image shows a Visual Studio Code editor window with a file named `differential_cryptanalysis.py` open. The script contains the following code:

```
12 | | differences.append(i)
13 |
14 | print("\nPositions with differences:", differences)
15 | print("Total Differences:", len(differences))
```

Below the editor is a terminal window. The terminal shows the execution of the script. It starts with a PowerShell prompt where the user sets the execution policy and runs the script. The script outputs the results of a probability analysis and then a differential cryptanalysis simulation.

```
PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\Activate.ps1")
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 7/probability_analysis.py"
===== PROBABILITY ANALYSIS =====
Enter successful outcomes: 3
Enter total outcomes: 8

Probability = 0.375
Percentage = 37.5 %
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY> & "e:\PROJECTS\ADVANCED CRYPTOGRAPHY\.venv\Scripts\python.exe" "e:/PROJECTS/ADVANCED CRYPTOGRAPHY/WEEK 7/differential_cryptanalysis.py"
===== DIFFERENTIAL CRYPTANALYSIS =====
Enter plaintext 1: HELLO
Enter plaintext 2: HELMO

Positions with differences: [3]
Total Differences: 1
(.venv) PS E:\PROJECTS\ADVANCED CRYPTOGRAPHY>
```

Figure 40: Differential cryptanalysis simulation showing plaintext differences.

CREATIVE TASK – Cryptanalysis Toolkit

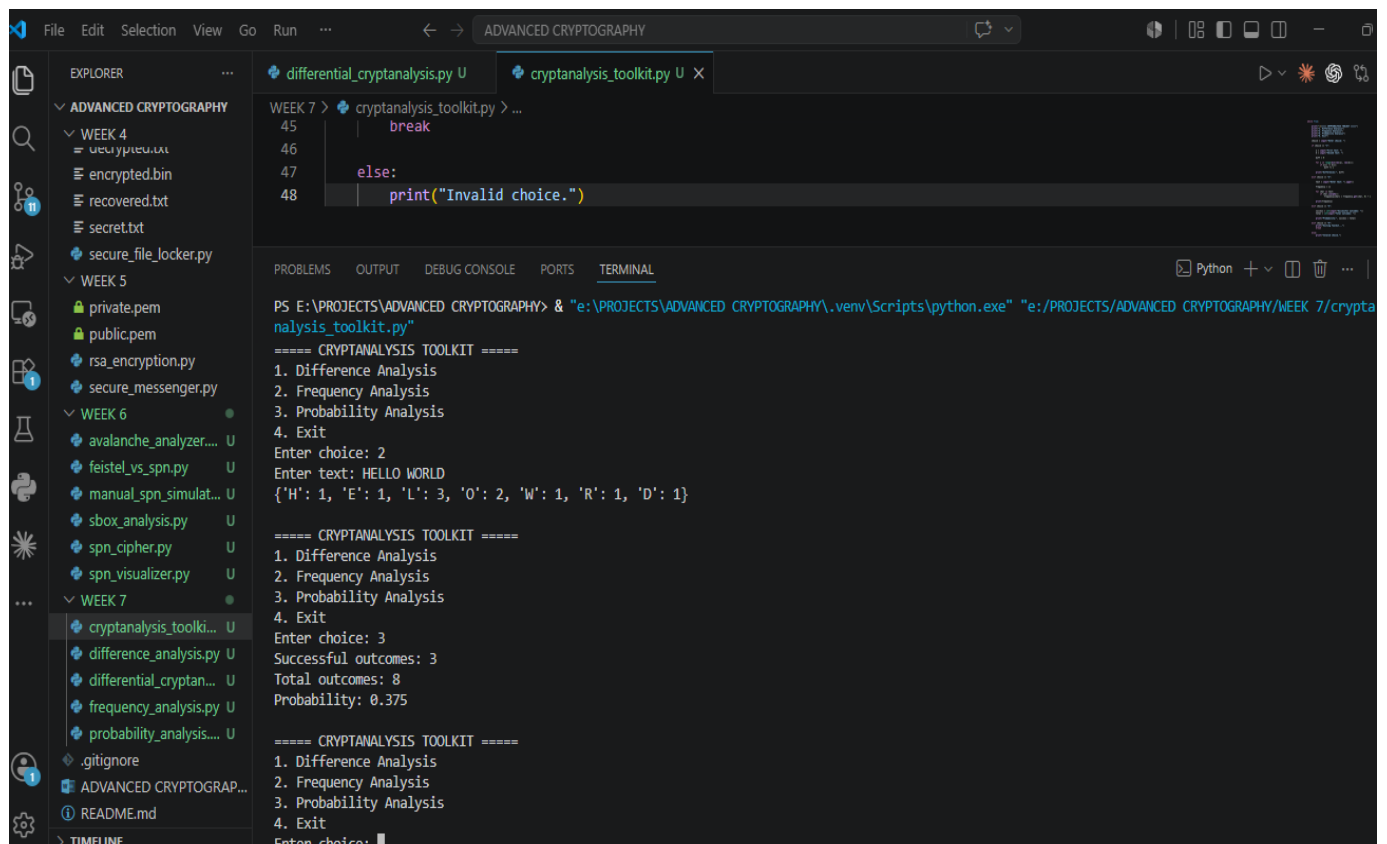


Figure 41: Cryptanalysis Toolkit integrating difference, frequency, and probability analysis techniques.

REFLECTION

Week 7 explored cryptanalysis techniques including difference analysis, frequency analysis, probability calculations, and differential cryptanalysis. The Cryptanalysis Toolkit combined these methods into a single application, improving understanding of how cryptographic systems can be evaluated and attacked.